VECTOR

Vector Code Analyzer

Technical Reference

Version 6.7.0

Authors	visamm, vislhe, vispnr, vistvi, visbfk
Status	Released

Document Information

History

Author	Date	Version	Remarks
vislhe	2020-03-10	4.0	Initial version
visamm	2020-05-08	4.1	Improve command-line interface.
vistvi	2020-12-09	5.7	Add Pygments license
visbfk	2021-06-01	5.18	Added description of warning messages
vistvi	2021-06-16	5.19	Editorial changes
visamm	2021-07-01	5.20	Updated command-line interface
visamm	2022-01-12	6.2.0	Add note that all files must be checked
vislhe	2022-03-14	6.3.0	Replace “MICROSAR” by “MICROSAR Classic and MICROSAR Adaptive”
visbfk	2022-03-25	6.5.0	Describe VCA execution for MICROSAR Adaptive
visbfk	2022-04-25	6.7.0	Extended required environment and adjusted target location mapping of VCA for MICROSAR Adaptive

Reference Documents

No.	Source	Title	Version
[1]	ISO	ISO/IEC 9899:1990, Programming languages -C	Second edition

Contents

1 Introduction **5**
1.1 Intended audience . . . 5
2 Functional Description **6**
2.1 Required Environment . . . 6
2.2 Restrictions . . . 6
2.3 Command Line Parameters (MICROSAR Classic) . . . 6
2.3.1 Option -I, –include . . . 7
2.3.2 Option -D, –define . . . 7
2.3.3 Option -p, –plugin-folder . . . 7
2.3.4 Option –open-report . . . 7
2.3.5 Option -r, –reportFile . . . 7
2.3.6 Command Line Usage (MICROSAR Classic) . . . 7
2.4 Command Line Parameters (MICROSAR Adaptive) . . . 7
2.4.1 Option –amsr-source-dir . . . 8
2.4.2 Option –build-dir . . . 8
2.4.3 Option -j . . . 8
2.4.4 Option –open-report . . . 8
2.4.5 Option -r, –reportFile . . . 8
2.4.6 Command Line Usage (MICROSAR Adaptive) . . . 8
3 Plugin File **9**
4 Analysis Report **10**
4.1 Structure . . . 10
4.1.1 Header . . . 10
4.1.2 Information about Environment . . . 10
4.1.3 Detailed Log Output . . . 10
4.2 Error Messages . . . 10
4.3 Warning Messages . . . 11
4.4 Steps if the Analysis Fails . . . 13
5 Integration MICROSAR Classic **14**
5.1 Deliverables . . . 14
5.2 DaVinci Configurator Pro 5 . . . 14
6 Integration MICROSAR Adaptive **16**
6.1 Deliverables . . . 16
6.2 Preparations for VCA analysis . . . 16
7 Third Party Libraries **18**
7.1 LLVM/Clang . . . 18

7.2 Z3 23
7.3 Python® 24
7.4 Python CMake File Api 25
7.5 Pygments 25
7.6 PyYAML 26
8 Contact **27**

1 Introduction

Vector Code Analyzer (VCA) is a command line tool delivered as part of MICROSAR Classic Safe and MICROSAR Adaptive Safe packages. VCA checks based on rules the absence of errors that can lead to a violation of the Freedom from Interference regarding memory (FFI w.r.t. memory) of static and generated source files of the BSW modules. The result is written to a HTML report. The report is part of the proof that the BSW modules fulfill the Freedom from Interference criteria.



Reference
For all required steps to be performed as part of the MICROSAR Classic Safe or MICROSAR Adaptive Safe integration see the project specific Safety Manual.

1.1 Intended audience

This document is relevant for developers who integrate MICROSAR Classic Safe or MICROSAR Adaptive Safe into their ECU.

2 Functional Description

This chapter describes the tool VCA and how it can be used to check the absence of errors - leading to violations of FFI w.r.t. memory - of the static and generated source files for MICROSAR Classic Safe and MICROSAR Adaptive Safe modules. VCA supports configuration via command line parameters. These are described in the following section together with the report produced by VCA.

2.1 Required Environment

VCA supports the following operating systems:

MICROSAR Classic:

- Windows 7 (64Bit)
- Windows 10 (64Bit)

MICROSAR Adaptive:

- Ubuntu 18.04 (64Bit)

2.2 Restrictions

VCA uses a compiler front end in order to compile the input source files. This compiler front end requires ISO C 90 [1] conform source code. Some target compilers implement specific language extensions which might prevent VCA from compiling the code successfully. The Vector BSW code does not contain such language extensions. However, these extensions may be included via customer header files. In such a case the customer shall take care that these language extensions are encapsulated via the preprocessor for the VCA execution. The corresponding preprocessor switches can be specified via the command line when calling VCA.

2.3 Command Line Parameters (MICROSAR Classic)

VCA is configured by means of command line parameters. This section describes the available command line parameters and their meaning. They are described in detail in the following sections.

Command Line Parameter	Description
Optional Parameters	
`-I, --include`	Includes all files found in the include path. A directory path includes all files directly in the directory. A wildcard path includes recursively all files directly under it and all folders beneath it.
`-D, -define`	Defines a new symbol. The define will directly be passed to the executable.
`-p, --plugin-folder`	Location of file specifying which plugins to load. By default, search in includes.
`--open-report`	After execution, opens the report in a browser.
`-r, --reportFile`	Specifies the path to the report.

2.3.1 Option -I, –include

Includes all files found in the include path. The path can be a filepath, e.g. /path/to/file.c, a directory path, e.g. /path/to, or a wildcard path, e.g. /path/to/*. A directory path includes all files directly in the directory. A wildcard path includes recursively all files directly under it and all folders beneath it.

2.3.2 Option -D, –define

Defines a new symbol. The define will directly be passed to the executable. A define can be formatted like `-D Foo`, `-D Foo=Bar` or `-D Foo=`.

2.3.3 Option -p, –plugin-folder

Specifies in which folder plugins should be searched. If omitted, plugins are searched in the includes.

2.3.4 Option –open-report

Opens the report in a browser after execution of VCA.

2.3.5 Option -r, –reportFile

Specifies the path to the report. If this parameter is missing, VCA writes the report to the current working directory with name “report.html”.

2.3.6 Command Line Usage (MICROSAR Classic)

This paragraph describes how VCA can be used with the documented parameters. Parameters which require a value are:

- `-I, --include`
- `-D, --define`
- `-p, --plugin-file`
- `-r, --reportFile`

These parameters require a value if they are specified on the command line. The value is separated from the identifier using a space.

For example:

```
CodeAnalyzer.exe -I includes\* -D DEBUG -p path\to\plugins
```

Example: Run VCA with minimum parameters

Assume that the root path of the delivery is `D:\delivery`. To run VCA with the minimum required parameters one can use:

```
CodeAnalyzer.exe -I D:\delivery\*
```

This will instruct VCA to scan the root path of the delivery recursively.

2.4 Command Line Parameters (MICROSAR Adaptive)

VCA is configured by means of command line parameters. This section describes the available command line parameters and their meaning. They are described in detail in the following sections.

Command Line Parameter	Description
Mandatory Parameters	
`--amsr-source-dir`	Path to the MICROSAR Adaptive source directory.
`--build-dir`	Path to the cmake build folder.
Optional Parameters	
`-j<N>`	Defines the number of jobs to run simultaneously for the VCA analysis.
`--open-report`	After execution, opens the report in a browser.
`-r, --reportFile`	Specifies the path to the report.

2.4.1 Option –amsr-source-dir

Path to the directory where the static source code files reside in. Choose the directory that contains, amongst others, the folders BSW and Generators.

2.4.2 Option –build-dir

The path to the generated cmake build project. In order to execute VCA it is necessary to generate the CMake build project using the provided VCA toolchain first (Cf. Integration MICROSAR Adaptive).

2.4.3 Option -j

Defines the number of jobs to run simultaneously for the VCA analysis. Half of the available cores are used by default.

2.4.4 Option –open-report

Opens the report in a browser after execution of VCA.

2.4.5 Option -r, –reportFile

Specifies the path to the report. If this parameter is missing, VCA writes the report to the current working directory with name “report.html”.

2.4.6 Command Line Usage (MICROSAR Adaptive)

VCA will analyze all targets and generated files relevant for the silent analysis as defined by the components. Start the analysis with the command line using minimum parameters:

```
CodeAnalyzer --amsr-source-dir <src-dir> --build-dir <build-dir>
```

It is possible to call VCA with optional parameters to change the report name or path, open the report directly after the analysis has finished or affect the analysis duration by choosing more or less cores to-be utilized for VCA analysis. Using all optional parameters the extended command line is the following:

```
CodeAnalyzer --amsr-source-dir <src-dir> --build-dir <build-dir>
-j<N> --report-file <report.html> --open-report
```

CodeAnalyzer will exit with a status code unequal to 0, if an error occured or the silent analysis finished with findings.

3 Plugin File

The plugin list file describes which plugins to execute. Each line contains the path to a plugin.

All relative plugins paths defined in the plugin-file are relative to the plugin file.

When a plugin is set up, essential values for the plugin to be analyzed are set. Essential values are the name and the files which must be analyzed or a list of defines.

The plugin list file and the plugins are contained in the delivery. They are searched in the paths provided by the `-p` option. The files must not be modified.



Note
The log output needs to be verified that all files have been correctly analyzed.



Note
For MICROSAR Adaptive no plugin list file is delivered. VCA will gather all plugins from the delivery.

4 Analysis Report

VCA generates a report file after performing its consistency checks. By default the report will be created in the current working directory as “report.html”. With -r or –reportFile the path for the report can be changed.

4.1 Structure

This report file has a simple structure to display the most important parts at a glance. The parts are described in the next sections.

4.1.1 Header

The header displays whether the VCA check was successful or not.

In case the check was successful, and no errors occurred, the header displays “Overall Check Result: Passed”.

If an error occurred, the header displays “Overall Check Result: Fail”.

Additionally, there may be a list of plugins which were skipped. This does not affect the overall result. In case one or more plugins were skipped, “Skipped Plugins: …” will be displayed directly under the overall check result.

This list of skipped plugins needs to be manually reviewed as to why they were skipped and if this has any impact on verdict regarding the absence of errors in the analyzed code.

For example, if a plugin was skipped because the module it is checking is not enabled and not used, this would most likely not affect the verdict. For more information refer to the project specific Safety Manual.

4.1.2 Information about Environment

The second section of the report displays some information about the environment of VCA. This includes for example the current windows user, and the number of found errors and warnings.

4.1.3 Detailed Log Output

The detailed log output contains general messages from VCA.



Note
The information found in the detailed log output is complex and not intended to be checked. The only relevant information for the user is the overall test result which states whether the check was successful or not. The detailed information becomes relevant if any errors occurred.

4.2 Error Messages

VCA emits general errors in the detailed log output section of the report file, plugin specific errors in the plugin section of the report file and displays them on the console. The report contains more details.

There exist five severity classes of VCA messages:

- Info,
- Note,
- Warning,

- Error,
- Fatal error.

Notes are emitted very often and are not of importance for the user of VCA but can help to trace a problem if one occurred.

Warnings indicate minor issues which are not relevant enough to issue an error. For example, if a source code file for a plugin was not found, a warning is emitted. An error is not emitted because the user may have disabled the module intentionally in the generator and thus the consistency of the project is not affected by this module. However, the plugin for which the source code file was not found is skipped and added to the list of skipped plugins in the report. This list needs to be reviewed manually.

An error can occur during the check of a BSW module if e.g. an invalid pointer access was detected. Errors do not prevent VCA from continuing its check and help the user to identify the underlying issue.

A fatal error indicates that a serious issue occurred that prevents VCA from performing the check.

The return value of the CodeAnalyzer.exe is influenced by these six classes of messages. The class of the highest occurred message defines the return value.

Message Class	Error Code
Info, Note, Warning in analysis findings	0
Error in analysis findings	1
Compiler Error	10
Linker Error	11
Fatal Error	99
Unknown Error	-1

For example, if an error occurred and no fatal errors of any kind the return value of VCA is 1. If only notes were emitted, VCA returns 0.

4.3 Warning Messages

The following table provides an overview over possible VCA messages with the severity class “warning” and possible actions that could be taken. Many actions refer the VCA Plugin file.

The location of the VCA Plugin file is: `<path to sip>\Generators\VCA\plugins`

Message Content	Background	Actions
warning: calling undefined function <function_name>	VCA only checks the BSW to which it is related. If the callee is a declaration without pointer arguments, this warning is reported.	Check the VCA Plugin file for all valid files concerning the SIP Delivery. The function cited in the warning message should not be defined within the listed sources (see “plugin.files”).

Message Content	Background	Actions
warning: function not used	If a function is not an entry point and is not called at least once, this warning is reported.	Check the VCA Plugin file for possible entry functions (see “plugin.entry_function”). Additional entry points should be listed in the VCA Plugin file.
warning: symbol <symbol> is not defined	If no definition or initialization of a symbol (or a function) was found, this warning is reported. This check includes external global variables if these variables are initialized in the analyzed sourced.	Check if the symbol contained in the warning message is defined within the analyzed sources listed in the VCA Plugin file (see “plugin.files”).
warning: control-flow location is unreachable	If a location is unreachable from all analyzed contexts, a warning is emitted.	Check if the call context of the location referred to in the warning message is part of the analyzed sources. All valid files concerning the SIP Delivery are listed in the VCA Plugin file (see “plugin.files”).
warning: case <case> is unreachable	If a transition of a switch statement is not taken from any context, this warning is reported.	Contact Vector providing the configuration that causes this warning message as this may relate a coverage issue.
warning: branch is unreachable	If a branch of an if-then-else statement is not taken from any context, this warning is reported.	Contact Vector providing the configuration that causes this warning message as this may relate a coverage issue.
warning: this is the only reachable branch	If a branch of an if-then-else statement is the only reachable branch (and the other branch is implicit), this warning is reported.	Contact Vector providing the configuration that causes this warning message as this may relate a coverage issue.
warning: specified invariant for global variable <var> is infeasible	This warning is reported for invariants of non-constant global variables if a contradiction between the formal specification and the specification exists. A variable’s value is regarded constant if the variable is declared with the modifier const or if no write access to the variable in the program exists.	Check the content of the formal specification against the specification of the global variable.

Message Content	Background	Actions
warning: memcpy reads | writes outside the allocated memory of <mem> when copying <amount of> bytes	The length of the object to copy and the actual copied content differentiate.	If this message occurs, please contact Vector.
warning: unknown pragma statement	The compiler does not understand the pragma of the target compiler.	This message can be ignored.

4.4 Steps if the Analysis Fails

In case the analysis reports a failure, Vector has to be informed about this issue. Please use the contact information found in the project specific Safety Manual.

5 Integration MICROSAR Classic

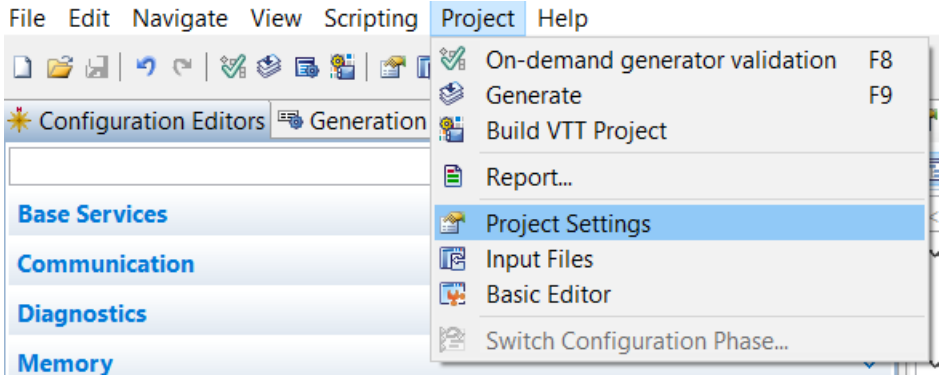
Since VCA checks the generated data it is convenient to run VCA automatically after the data is generated. DaVinci Configurator Pro 5 supports external generation steps to allow easy integration.

5.1 Deliverables

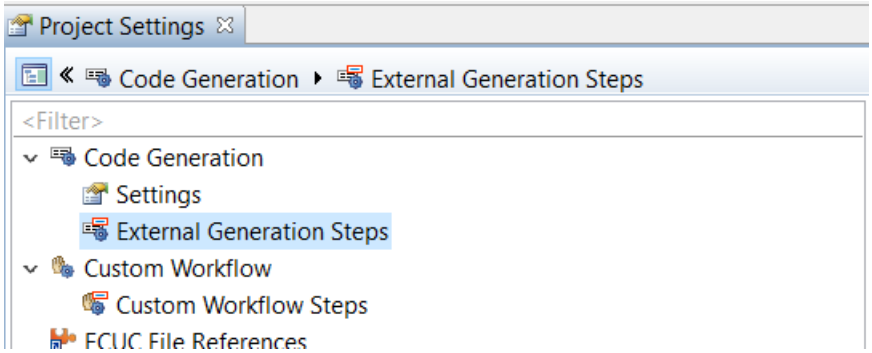
The following table describes which parts are delivered with the VCA tool and where to find them.

Deliverable	Location
Vector Code Analyzer	.\Generators\VCA\CodeAnalyzer.exe
Technical Reference	.\Doc\TechnicalReferences\TechnicalReference_VCA.pdf

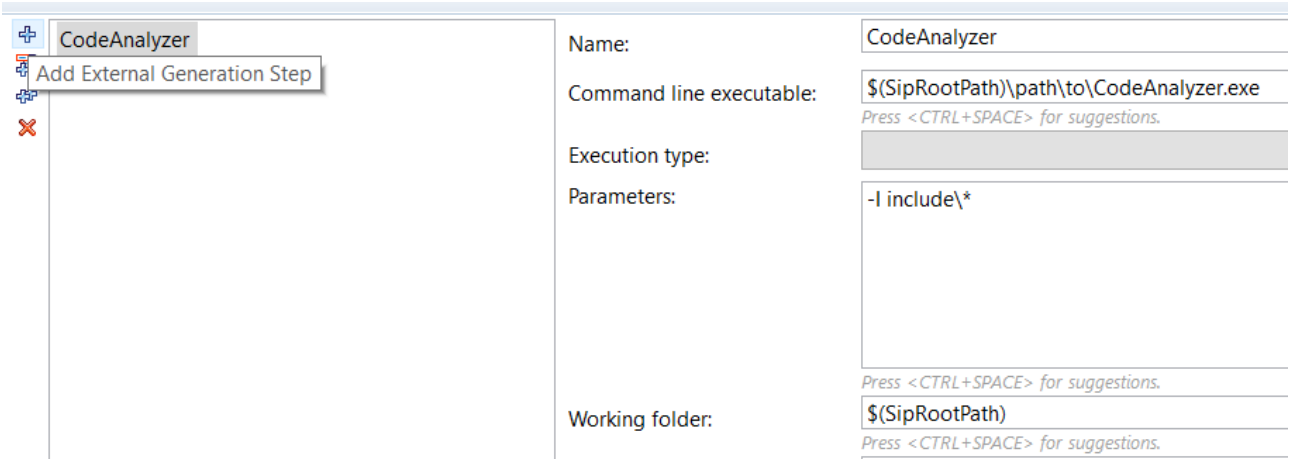
5.2 DaVinci Configurator Pro 5



Start DaVinci Configurator Pro 5 and select the menu “Project”. Next select the menu item “Project Settings”.



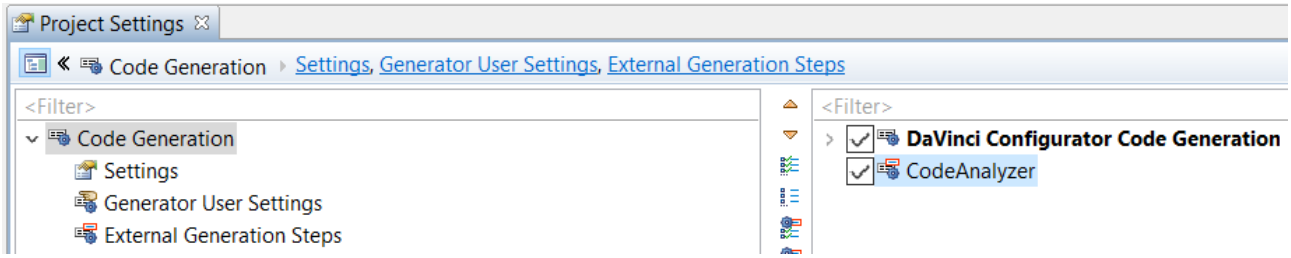
To add a new external generation step, select “External Generation Steps”. This will display the following window:



Click on the Add button with the “+” symbol and enter the VCA path and command line arguments.

Note
It is required to set a working directory for a post generation step.

Now the external generation step needs to be configured to be run after the DaVinci Generators. To configure this click on the item “Code Generation” in the location bar in the same window.



Now select the VCA Generation Step and enable it by checking the check box in front of it. Additionally VCA should be run after DaVinci Configurator Pro generated the data. Therefore it is necessary to move it after the DaVinci Code Generation using the Down button with the “▼” symbol.

Now VCA will be automatically executed after the DaVinci Configurator Pro has generated the data.

Note
VCA will also be executed if the data was not successfully generated.

6 Integration MICROSAR Adaptive

This chapter describes the usage of the VCA for applications using MICROSAR Adaptive. Before starting the analysis with VCA it is necessary to generate the CMake build project using the VCA toolchain file and run the code generators to make sure all files defined to-be analyzed are available.

6.1 Deliverables

The following table describes which parts are delivered with the VCA tool and where to find them.

Deliverable	Location
Vector Code Analyzer	./Tools/VCA/CodeAnalyzer-<version>.deb
Technical Reference	./Doc/TechnicalReferences/TechnicalReference_VCA.pdf

6.2 Preparations for VCA analysis

The following steps are assumed to be executed in the source directory of the application using MICROSAR Adaptive. If executed in other directories, the paths must be adapted accordingly.

If the CMakePresets.json file has not been created yet, add it to the project source directory. Add a CMake preset for VCA to the CMakePresets.json file.

```
{
  [...]
  "configurePresets": [
    [...]
    {
      "name": "vca_linux_x86_64",
      "inherits": "unix-makefiles",
      "description": "Debug build with Vector Code Analyzer for Linux x86-64",
      "binaryDir": "build/vca_linux_x86_64",
      "cacheVariables": {
        "CCACHE_ENABLE": "OFF",
        "CMAKE_BUILD_TYPE": "Release",
        "CMAKE_LINK_WHAT_YOU_USE": false,
        "CMAKE_INSTALL_PREFIX": "${AMSR_SRC_DIR}/build/install/linux_vca",
        "CMAKE_PREFIX_PATH": "/opt/VectorCodeAnalyzer",
        "CMAKE_TOOLCHAIN_FILE": {
          "type": "FILEPATH",
          "value": "$env{AMSR_SRC_DIR}/CMake/Toolchain/vca_linux_x86_64.cmake"
        },
        "Python3_ROOT": null
      }
    }
  ]
}
```

Generate the CMake build project using the VCA toolchain provided with TACO:

```
$ export AMSR_SRC_DIR=/path/to/adaptive-microsar
$ cmake -S . -C ${AMSR_SRC_DIR}/CMakeConfig/linux_vca.cmake --preset=vca_linux_x86_64
```

Since VCA checks the generated data it is necessary to run VCA after the data is generated. Run the file generation with the DaVinci Adaptive Configurator and already built generators:

```
$ cd build/vca_linux_x86_64
$ ctest -R '\.DvACfg\.Generate'
$ cmake .
```

Install the CodeAnalyzer executable from from the delivered CodeAnalyzer debian package. Please make sure to install the provided version and remove previous versions beforehand if required. Start the VCA analysis as described above.

```
CodeAnalyzer --amsr-source-dir $AMSR_SRC_DIR --build-dir .
```

7 Third Party Libraries

7.1 LLVM/Clang

```
==============================================================================
The LLVM Project is under the Apache License v2.0 with LLVM Exceptions:
==============================================================================

                                 Apache License
                           Version 2.0, January 2004
                           http://www.apache.org/licenses/

    TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

    1. Definitions.

    "License" shall mean the terms and conditions for use, reproduction,
    and distribution as defined by Sections 1 through 9 of this document.

    "Licensor" shall mean the copyright owner or entity authorized by
    the copyright owner that is granting the License.

    "Legal Entity" shall mean the union of the acting entity and all
    other entities that control, are controlled by, or are under common
    control with that entity. For the purposes of this definition,
    "control" means (i) the power, direct or indirect, to cause the
    direction or management of such entity, whether by contract or
    otherwise, or (ii) ownership of fifty percent (50%) or more of the
    outstanding shares, or (iii) beneficial ownership of such entity.

    "You" (or "Your") shall mean an individual or Legal Entity
    exercising permissions granted by this License.

    "Source" form shall mean the preferred form for making modifications,
    including but not limited to software source code, documentation
    source, and configuration files.

    "Object" form shall mean any form resulting from mechanical
    transformation or translation of a Source form, including but
    not limited to compiled object code, generated documentation,
    and conversions to other media types.

    "Work" shall mean the work of authorship, whether in Source or
    Object form, made available under the License, as indicated by a
    copyright notice that is included in or attached to the work
    (an example is provided in the Appendix below).

    "Derivative Works" shall mean any work, whether in Source or Object
    form, that is based on (or derived from) the Work and for which the
    editorial revisions, annotations, elaborations, or other modifications
    represent, as a whole, an original work of authorship. For the purposes
    of this License, Derivative Works shall not include works that remain
    separable from, or merely link (or bind by name) to the interfaces of,
    the Work and Derivative Works thereof.

    "Contribution" shall mean any work of authorship, including
    the original version of the Work and any modifications or additions
    to that Work or Derivative Works thereof, that is intentionally
    submitted to Licensor for inclusion in the Work by the copyright owner
    or by an individual or Legal Entity authorized to submit on behalf of
```

the copyright owner. For the purposes of this definition, "submitted"
means any form of electronic, verbal, or written communication sent
to the Licensor or its representatives, including but not limited to
communication on electronic mailing lists, source code control systems,
and issue tracking systems that are managed by, or on behalf of, the
Licensor for the purpose of discussing and improving the Work, but
excluding communication that is conspicuously marked or otherwise
designated in writing by the copyright owner as "Not a Contribution."

"Contributor" shall mean Licensor and any individual or Legal Entity
on behalf of whom a Contribution has been received by Licensor and
subsequently incorporated within the Work.

1. Grant of Copyright License. Subject to the terms and conditions of
this License, each Contributor hereby grants to You a perpetual,
worldwide, non-exclusive, no-charge, royalty-free, irrevocable
copyright license to reproduce, prepare Derivative Works of,
publicly display, publicly perform, sublicense, and distribute the
Work and such Derivative Works in Source or Object form.

1. Grant of Patent License. Subject to the terms and conditions of
this License, each Contributor hereby grants to You a perpetual,
worldwide, non-exclusive, no-charge, royalty-free, irrevocable
(except as stated in this section) patent license to make, have made,
use, offer to sell, sell, import, and otherwise transfer the Work,
where such license applies only to those patent claims licensable
by such Contributor that are necessarily infringed by their
Contribution(s) alone or by combination of their Contribution(s)
with the Work to which such Contribution(s) was submitted. If You
institute patent litigation against any entity (including a
cross-claim or counterclaim in a lawsuit) alleging that the Work
or a Contribution incorporated within the Work constitutes direct
or contributory patent infringement, then any patent licenses
granted to You under this License for that Work shall terminate
as of the date such litigation is filed.

1. Redistribution. You may reproduce and distribute copies of the
Work or Derivative Works thereof in any medium, with or without
modifications, and in Source or Object form, provided that You
meet the following conditions:

(a) You must give any other recipients of the Work or
    Derivative Works a copy of this License; and

(b) You must cause any modified files to carry prominent notices
    stating that You changed the files; and

(c) You must retain, in the Source form of any Derivative Works
    that You distribute, all copyright, patent, trademark, and
    attribution notices from the Source form of the Work,
    excluding those notices that do not pertain to any part of
    the Derivative Works; and

(d) If the Work includes a "NOTICE" text file as part of its
    distribution, then any Derivative Works that You distribute must
    include a readable copy of the attribution notices contained
    within such NOTICE file, excluding those notices that do not
    pertain to any part of the Derivative Works, in at least one
    of the following places: within a NOTICE text file distributed
    as part of the Derivative Works; within the Source form or
    documentation, if provided along with the Derivative Works; or,

within a display generated by the Derivative Works, if and
wherever such third-party notices normally appear. The contents
of the NOTICE file are for informational purposes only and
do not modify the License. You may add Your own attribution
notices within Derivative Works that You distribute, alongside
or as an addendum to the NOTICE text from the Work, provided
that such additional attribution notices cannot be construed
as modifying the License.

You may add Your own copyright statement to Your modifications and
may provide additional or different license terms and conditions
for use, reproduction, or distribution of Your modifications, or
for any such Derivative Works as a whole, provided Your use,
reproduction, and distribution of the Work otherwise complies with
the conditions stated in this License.

1. Submission of Contributions. Unless You explicitly state otherwise,
any Contribution intentionally submitted for inclusion in the Work
by You to the Licensor shall be under the terms and conditions of
this License, without any additional terms or conditions.
Notwithstanding the above, nothing herein shall supersede or modify
the terms of any separate license agreement you may have executed
with Licensor regarding such Contributions.

1. Trademarks. This License does not grant permission to use the trade
names, trademarks, service marks, or product names of the Licensor,
except as required for reasonable and customary use in describing the
origin of the Work and reproducing the content of the NOTICE file.

1. Disclaimer of Warranty. Unless required by applicable law or
agreed to in writing, Licensor provides the Work (and each
Contributor provides its Contributions) on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or
implied, including, without limitation, any warranties or conditions
of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A
PARTICULAR PURPOSE. You are solely responsible for determining the
appropriateness of using or redistributing the Work and assume any
risks associated with Your exercise of permissions under this License.

1. Limitation of Liability. In no event and under no legal theory,
whether in tort (including negligence), contract, or otherwise,
unless required by applicable law (such as deliberate and grossly
negligent acts) or agreed to in writing, shall any Contributor be
liable to You for damages, including any direct, indirect, special,
incidental, or consequential damages of any character arising as a
result of this License or out of the use or inability to use the
Work (including but not limited to damages for loss of goodwill,
work stoppage, computer failure or malfunction, or any and all
other commercial damages or losses), even if such Contributor
has been advised of the possibility of such damages.

1. Accepting Warranty or Additional Liability. While redistributing
the Work or Derivative Works thereof, You may choose to offer,
and charge a fee for, acceptance of support, warranty, indemnity,
or other liability obligations and/or rights consistent with this
License. However, in accepting such obligations, You may act only
on Your own behalf and on Your sole responsibility, not on behalf
of any other Contributor, and only if You agree to indemnify,
defend, and hold each Contributor harmless for any liability
incurred by, or claims asserted against, such Contributor by reason
of your accepting any such warranty or additional liability.

```
    END OF TERMS AND CONDITIONS

    APPENDIX: How to apply the Apache License to your work.

    To apply the Apache License to your work, attach the following
    boilerplate notice, with the fields enclosed by brackets "[]"
    replaced with your own identifying information. (Don't include
    the brackets!)  The text should be enclosed in the appropriate
    comment syntax for the file format. We also recommend that a
    file or class name and description of purpose be included on the
    same "printed page" as the copyright notice for easier
    identification within third-party archives.

    Copyright [yyyy] [name of copyright owner]

    Licensed under the Apache License, Version 2.0 (the "License");
    you may not use this file except in compliance with the License.
    You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

    Unless required by applicable law or agreed to in writing, software
    distributed under the License is distributed on an "AS IS" BASIS,
    WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
    See the License for the specific language governing permissions and
    limitations under the License.


---- LLVM Exceptions to the Apache 2.0 License ----

As an exception, if, as a result of your compiling your source code, portions
of this Software are embedded into an Object form of such source code, you
may redistribute such embedded portions in such Object form without complying
with the conditions of Sections 4(a), 4(b) and 4(d) of the License.

In addition, if you combine or link compiled forms of this Software with
software that is licensed under the GPLv2 ("Combined Software") and if a
court of competent jurisdiction determines that the patent provision (Section
3), the indemnity provision (Section 9) or other Section of the License
conflicts with the conditions of the GPLv2, you may retroactively and
prospectively choose to deem waived or otherwise exclude such Section(s) of
the License, but only in their entirety and only with respect to the Combined
Software.

==============================================================================
Software from third parties included in the LLVM Project:
==============================================================================
The LLVM Project contains third party software which is under different license
terms. All such code will be identified clearly using at least one of two
mechanisms:
1) It will be in a separate directory tree with its own `LICENSE.txt` or
`LICENSE` file at the top containing the specific license and restrictions
which apply to that software, or
1) It will contain specific license and restriction terms at the top of every
file.

==============================================================================
Legacy LLVM License (https://llvm.org/docs/DeveloperPolicy.html#legacy):
==============================================================================
University of Illinois/NCSA
```

Open Source License

Copyright (c) 2003-2019 University of Illinois at Urbana-Champaign.
All rights reserved.

Developed by:

    LLVM Team

    University of Illinois at Urbana-Champaign

    http://llvm.org

Permission is hereby granted, free of charge, to any person obtaining a copy of
this software and associated documentation files (the "Software"), to deal with
the Software without restriction, including without limitation the rights to
use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies
of the Software, and to permit persons to whom the Software is furnished to do
so, subject to the following conditions:

    * Redistributions of source code must retain the above copyright notice,
    this list of conditions and the following disclaimers.

    * Redistributions in binary form must reproduce the above copyright notice,
    this list of conditions and the following disclaimers in the
    documentation and/or other materials provided with the distribution.

    * Neither the names of the LLVM Team, University of Illinois at
    Urbana-Champaign, nor the names of its contributors may be used to
    endorse or promote products derived from this Software without specific
    prior written permission.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS
FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
CONTRIBUTORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS WITH THE
SOFTWARE.

7.2 Z3

Z3
Copyright (c) Microsoft Corporation
All rights reserved.
MIT License

Permission is hereby granted, free of charge, to any person obtaining a
copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, including
without limitation the rights to use, copy, modify, merge, publish,
distribute, sublicense, and/or sell copies of the Software, and to
permit persons to whom the Software is furnished to do so, subject to
the following conditions:

The above copyright notice and this permission notice shall be included
in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED *AS IS*, WITHOUT WARRANTY OF ANY KIND, EXPRESS
OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT,
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

7.3 Python®

```
PYTHON SOFTWARE FOUNDATION LICENSE VERSION 2
--------------------------------------------

1. This LICENSE AGREEMENT is between the Python Software Foundation
("PSF"), and the Individual or Organization ("Licensee") accessing and
otherwise using this software ("Python") in source or binary form and
its associated documentation.

2. Subject to the terms and conditions of this License Agreement, PSF
hereby grants Licensee a nonexclusive, royalty-free, world-wide license
to reproduce, analyze, test, perform and/or display publicly, prepare
derivative works, distribute, and otherwise use Python alone or in any
derivative version, provided, however, that PSF's License Agreement and
PSF's notice of copyright, i.e., "Copyright (c) 2001, 2002, 2003, 2004,
2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016,
2017, 2018, 2019, 2020 Python Software Foundation;
All Rights Reserved" are retained in Python alone or in any derivative
version prepared by Licensee.

3. In the event Licensee prepares a derivative work that is based on
or incorporates Python or any part thereof, and wants to make
the derivative work available to others as provided herein, then
Licensee hereby agrees to include in any such work a brief summary of
the changes made to Python.

4. PSF is making Python available to Licensee on an "AS IS"
basis.  PSF MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR
IMPLIED.  BY WAY OF EXAMPLE, BUT NOT LIMITATION, PSF MAKES NO AND
DISCLAIMS ANY REPRESENTATION OR WARRANTY OF MERCHANTABILITY OR FITNESS
FOR ANY PARTICULAR PURPOSE OR THAT THE USE OF PYTHON WILL NOT
INFRINGE ANY THIRD PARTY RIGHTS.

5. PSF SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF PYTHON
FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS
A RESULT OF MODIFYING, DISTRIBUTING, OR OTHERWISE USING PYTHON,
OR ANY DERIVATIVE THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a material
breach of its terms and conditions.

7. Nothing in this License Agreement shall be deemed to create any
relationship of agency, partnership, or joint venture between PSF and
Licensee.  This License Agreement does not grant permission to use PSF
trademarks or trade name in a trademark sense to endorse or promote
products or services of Licensee, or any third party.

8. By copying, installing or otherwise using Python, Licensee
agrees to be bound by the terms and conditions of this License
Agreement.
```

7.4 Python CMake File Api

Copyright (c) 2020 Maarten

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

7.5 Pygments

Copyright (c) 2006-2020 by the respective authors (see AUTHORS file).
All rights reserved.

Redistribution and use in source and binary forms, with or without
modification, are permitted provided that the following conditions are
met:

* Redistributions of source code must retain the above copyright
notice, this list of conditions and the following disclaimer.

* Redistributions in binary form must reproduce the above copyright
notice, this list of conditions and the following disclaimer in the
documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR
A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL,
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE,
DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY
THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

7.6 PyYAML

Copyright (c) 2017-2021 Ingy döt Net
Copyright (c) 2006-2016 Kirill Simonov

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com